

Windows Integration into the Green Metrics Tool: Energy Measurement of Native Windows Applications via RAPL

Maximilian Jahns¹, Geerd-Dietger Hoffmann², Dennis Junger¹, and Volker Wohlgemuth¹

Abstract: The Green Metrics Tool (GMT) profiles energy and performance for Linux and macOS, but previously had no Windows support. This work integrates Intel RAPL measurement into GMT for native Windows applications, comparing two hosting designs that leave GMT's core components unmodified: a WSL2-bridged prototype in which the GMT runner executes inside WSL2, and a native Windows design orchestrating measurement containers via Docker Desktop. Both are validated with ten-run reproducibility studies of a Blender 5.1.1 (*Lone Monk*, Cycles CPU, 1440 × 1080) benchmark. Total-energy coefficients of variation stay at or below 2.7 % in every domain: the native design yields 1.272 ± 0.028 Wh for `cpu_package` (CV = 2.2 %) and the WSL2-bridged design 1.166 ± 0.019 Wh (CV = 1.6 %); a second acquisition path (EMI) reproduces the native figure to within 0.1 %. The identical Windows workload differs by 9.1 % in energy and 10.9 % in render duration between the two hosting designs – reported as observed in the tested configuration and not separable from a time-of-day confound. The MSR-reading driver layer is adopted from the Scaphandre project; the GMT integration and reproducibility evaluation are original to this work, which supports UN Sustainable Development Goals 9 and 12 by making the energy footprint of Windows software measurable within a certification-anchored framework.

Keywords: RAPL, Windows, Green Metrics Tool, energy measurement, sustainable software, reproducibility

1 Introduction

Environmental management information systems (EMIS) help organizations systematically record, evaluate, and translate the environmental impact of their processes into decisions [Wo05], increasingly including the energy consumption of software and IT infrastructure [Na11]. Crucially, an EMIS is not only a lens onto other software – it is itself software, operated and maintained, and should therefore be subject to the same evaluation standards it demands of other digital systems. A heterogeneous tool ecosystem has established itself for software-related energy measurement, but this diversity makes comparability across systems and platforms harder: tools differ in resolution, instrumentation level, and methodological validity, and many deliver only retrospective figures without robust comparisons [OBR20; RT24].

This is precisely where the Green Metrics Tool (GMT) comes in. Rather than providing yet another isolated measurement tool, GMT anchors its methodology in a recognized

¹ HTW Berlin – University of Applied Sciences, Environmental Informatics Unit, Wilhelminenhofstraße 75A, 12459 Berlin, Germany,

² Green Coding Solutions GmbH, Berlin, Germany,

certification framework – the Blue Angel eco-label for resource- and energy-conscious software products. Through defined product groups, audited award criteria, and standardized test scenarios, it creates a basis for comparison that individual measurement tools cannot offer on their own: what matters is not an isolated benchmark figure, but the reproducibility and transferability of measurements across software, versions, and operating environments [NGK20].

That robustness, however, has so far been confined to Linux and macOS, while Windows-based workstations and engineering applications retain high practical relevance in corporate IT landscapes. The obstacle is technical: although Intel Running Average Power Limit (RAPL) is available at the processor level independent of the operating system, Windows does not expose the underlying Model Specific Registers (MSRs) to applications, and access requires a signed kernel driver [Sc19]. Existing tools such as Scaphandre [Sc24] enable RAPL access under Windows in principle, but offer no interface to GMT’s metric-provider architecture. This work closes that gap and makes the following contributions:

- Two architectures for RAPL energy measurement on Windows within the GMT framework – which had no such capability before – are presented and compared: a WSL2-bridged design and a native Windows design that orchestrates measurement containers via Docker Desktop, neither modifying GMT’s core components. The MSR-reading kernel driver and the core acquisition loop are adopted from the open-source Scaphandre project [Sc24]; the contribution of this work is the integration of that access path into GMT.
- A GMT metric provider built around this driver, extended with environment-variable-based configuration and integrated driver-health monitoring.
- A reproducibility evaluation ($n = 10$) of both designs, cross-checked against a second acquisition path (EMI).

2 Background and Related Work

Technical measurement approaches build on Intel RAPL interfaces, ACPI, hardware counters, kernel instrumentation, or container-related telemetry [RT24]. Several tools address software-related energy measurement without being tied to a certification methodology such as the Blue Angel. PowerAPI [Fi24] provides a middleware layer for process-level RAPL measurement via the MSR kernel module, with limited Windows support. Scaphandre [Sc24] is an agent-based tool with RAPL support on Linux and Windows, using a kernel driver (ScaphandreDrv) for MSR access and exporting to Prometheus, but without GMT integration. EnergiBridge [SCD24] wraps cross-platform measurement around arbitrary commands, including a Windows backend, but operates standalone. GMT [Ho25] closes this methodological gap with standardized metrics tied directly to the Blue Angel methodology; it orchestrates Docker containers, structures runs into phases, and stores data from a configurable set of providers in a PostgreSQL database, with a frontend for

visualization and impact conversion [Gr24]. GMT contains providers for Linux RAPL (`CpuEnergyRaplMsrComponentProvider`), machine-wide PSU measurement, and model-based estimation, but no Windows-native RAPL provider existed prior to this work. To the best of our knowledge, no prior work integrates Windows RAPL measurements natively into a Docker-orchestrated energy benchmarking framework.

Intel RAPL, introduced with Sandy Bridge, provides cumulative energy counters via MSRs for the domains `cpu_package`, `cpu_cores`, `cpu_gpu`, `dram`, and `psys` (platform energy, from Skylake onward), each incrementing in units defined by MSR `0x606` (typically $\approx 15.3 \mu\text{J}$) [RT24]. Under Linux these are reachable via the `Powercap` class or directly via `MSR`; under Windows no equivalent user-space interface exists, so access requires a kernel driver.

3 Architecture

Closing this gap required bridging Windows’ kernel-level MSR access with GMT’s Python-based metric-provider framework. Two designs were explored, both introduced by this work and summarized in Table 1. They share the same driver and acquisition-binary layers (Section 3.1) and differ only in how the GMT runner is hosted; both are evaluated with identical $n = 10$ studies, enabling direct comparison (Section 4.4).

Tab. 1: Design 1 (WSL2-bridged) vs. Design 2 (native Windows)

	Design 1 (WSL2)	Design 2 (native)
GMT runtime hosted in	WSL2/Ubuntu (Linux)	Windows user space
Reaches acquisition binary via	Interop. bridge (Unix socket)	Direct subprocess call
Workload (<code>blender.exe</code>) runs as	Windows host process	Windows host process
GMT runner requires WSL2	Yes	No
Evaluated in this work	$n = 10$ (Table 3)	$n = 10$ (Table 4)

3.1 Implementation

The Windows RAPL integration is structured into four layers: (1) the `ScaphandreDrv.sys` kernel driver, (2) the `rapl_reader.exe` acquisition binary, (3) GMT’s `provider.py` metric provider, and (4) `runner.py` with the PostgreSQL database, handling phase control, orchestration, and storage. Layers 1–2 are adopted from the open-source Windows RAPL driver maintained by the Scaphandre project [Sc24]; the contribution of this work begins at Layer 3.

The `ScaphandreDrv.sys` kernel driver registers a Windows device object under `\Device\ScaphandreDriver`, exposing a single IOCTL code that accepts an MSR register address and CPU index and returns the 64-bit raw MSR value. The second layer,

`rapl_reader.exe`, is a C application for Windows x64 that opens this device handle and continuously reads RAPL values at configurable intervals. Energy is obtained by differencing the cumulative RAPL counters between consecutive reads, with counter wraparound handled explicitly, so the reported energies are independent of sampling jitter. Four mechanisms ensure measurement quality: `QueryPerformanceCounter()` anchors a wall-clock Unix timestamp at start, producing monotonic microsecond timestamps compatible with GMT’s timing mechanism; `timeBeginPeriod(1)` sets the Windows timer resolution to 1 ms so that `Sleep(99)` sleeps ≈ 99 ms instead of ≈ 15.6 ms, with a deadline-based mechanism compensating measurement time per round; a detection pass at startup disables unsupported domains; and the PKG domain serves as a driver-health canary, exiting after 5 consecutive IOCTL failures – which did not occur in any reported run.

The next layer integrates GMT’s metric provider: `rapl_reader.exe` is started as a native Windows subprocess, with stdout redirected to a file that GMT’s `BaseProvider` parses after the run. In Design 1 that subprocess is launched from within WSL2 through the interoperability layer and its output consumed by the WSL2-hosted runner; in Design 2 it is launched directly from `provider.py`. The new provider extends `BaseMetricProvider`, overriding `check_system()` (verifies driver availability) and `start_profiling()` (constructs and starts the Windows command with output redirection).

4 Evaluation – Blender Rendering

4.1 Experimental Setup

Blender 5.1.1 (build 2026-04-14) under Windows 11 was used as the test application: an open-source 3D creation suite producing reproducible, CPU-intensive workloads. The scene, *Lone Monk*, has complex geometry with several hundred objects. DRAM and CPU/GPU domains are not supported on the test system and are automatically disabled. In both designs the workload is the same `blender.exe` Windows binary executing as a host process; the designs differ only in whether it is invoked through the WSL2 interoperability layer (Design 1) or directly (Design 2). In both designs the RAPL values are acquired by `rapl_reader.exe` through the `ScaphandreDrv.sys` driver, so the two arms share an identical acquisition path. Table 2 shows the common configuration. GMT structures each run into phases: [BASELINE] (system at rest before any container starts, the reference idle level), [INSTALLATION], [BOOT], [IDLE] (containers up, no workload), [Blender Render], and [REMOVE]. Unless stated otherwise, reported energies refer to [Blender Render]; net energy subtracts a duration-scaled [BASELINE] idle baseline.

4.2 Design 1: WSL2-bridged

Design 1 was evaluated under an $n = 10$ repeated-run protocol. Table 3 shows the results. The CV of 1.6 % reflects a quiet execution environment: the idle baseline is typically ≈ 4.1 W

Tab. 2: Configuration parameters of the Blender benchmark experiment

Parameter	Value
Renderer	Cycles (CPU-only, no GPU rendering)
Scene	Lone Monk
Samples	64 spp (samples per pixel)
Resolution	1440 × 1080 px
Render duration	≈266 s (Design 1/WSL2), ≈295 s (Design 2/Windows)
Operating system	Windows 11; WSL2 Ubuntu 22.04 for Design 1, Docker Desktop for Design 2
CPU	Intel Core i7-1185G7, Turbo Boost enabled
RAM	16 GB DDR4
RAPL sampling rate	99 ms
Measured domains	cpu_package, cpu_cores, psys

Tab. 3: Design 1 – WSL2-bridged, $n = 10$ repeated runs (Scaphandre/MSR acquisition path). Render-phase energy, mean $\pm$ SD. Net-energy CVs are elevated by one anomalous run (476c3d76) in which the Windows host showed unusually high background activity during [BASELINE] (≈ 8.5 W vs. ≈ 4.1 W typical).

Domain	Total (Wh)	Net (Wh)	CV total	CV net
cpu_cores	0.890 ± 0.012	0.798 ± 0.070	1.3 %	8.8 %
cpu_package	1.166 ± 0.019	0.825 ± 0.102	1.6 %	12.4 %
psys	2.337 ± 0.044	1.018 ± 0.134	1.9 %	13.2 %

(mean ≈ 4.5 W including run 476c3d76) and the render duration highly consistent at 266 ± 2 s (CV = 0.8 %).

4.3 Design 2: Windows Native

Design 2 was evaluated under the same $n = 10$ protocol on the same machine. Table 4 shows total and net render-phase energy per domain. Total-energy CVs (1.4–2.7 %) are

Tab. 4: Design 2 – Windows native, $n = 10$ repeated runs. Render-phase energy, mean $\pm$ SD. Scaphandre/MSR acquisition path unless noted; the EMI row is a consistency check on cpu_package and shares the same baseline term in the net column.

Domain	Total (Wh)	Net (Wh)	CV total	CV net
cpu_cores (Scaphandre)	0.951 ± 0.013	0.766 ± 0.057	1.4 %	7.4 %
cpu_package (Scaphandre)	1.272 ± 0.028	0.797 ± 0.072	2.2 %	9.0 %
cpu_package (EMI)	1.271 ± 0.028	0.797 ± 0.072	2.2 %	9.0 %
psys (Scaphandre)	2.579 ± 0.070	0.992 ± 0.113	2.7 %	11.4 %

slightly higher than for Design 1, driven by run-to-run variation in [BASELINE] idle power

rather than in render-induced consumption itself: Windows idle power fluctuated between 4.9 and 7.9 W across the ten runs, consistent with background services varying their activity between sessions. Net-energy variability is higher (7.4–11.4 %) for the same reason, and the render duration is less stable at 295 ± 9 s (CV = 3.0 %) vs. 266 ± 2 s. The first run shows the lowest total energy across all domains, plausibly a Turbo-Boost cold-start effect. A second acquisition path – EMI (Energy Meter Interface, an inbox Windows 10+ driver interface) [msemi2024] – yields 1.271 ± 0.028 Wh for `cpu_package`, agreeing with Scaphandre/MSR to within 0.1 %. On this platform EMI is understood to surface the same RAPL package counter, so the agreement is a consistency check on the software pipeline, not an external validation of the counter; that the two channels are physically identical was not verified.

4.4 Comparison of the Two Hosting Designs

Figure 1 shows the mean power traces (`cpu_package`) for both designs across their respective $n = 10$ runs, aligned to render start. Figure 2 shows the mean energy per GMT phase. Together they reveal three systematic differences plus one methodological caveat. Because the workload binary, the acquisition path and the hardware are identical in both arms, these differences reflect the conditions under which the measurement run was carried out, not a property of the workload itself.

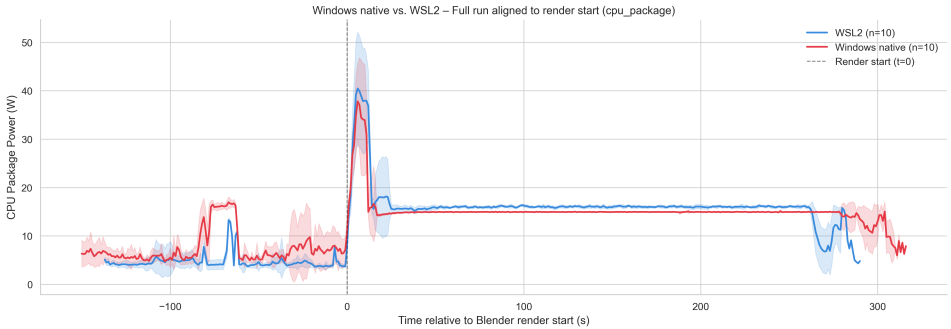


Fig. 1: Mean `cpu_package` power over time, aligned to render start – Design 1/WSL2 (blue, $n = 10$) vs. Design 2/Windows native (red, $n = 10$). Shading denotes ± 1 SD. The Windows BASELINE level is visibly higher, and the render phase extends further to the right.

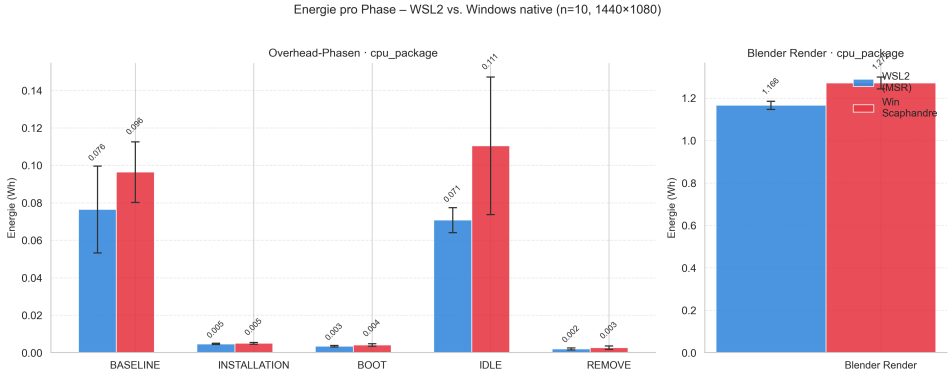


Fig. 2: Mean energy per GMT phase – Design 1/WSL2 (blue) vs. Design 2/Windows native (red), `cpu_package` ($n = 10$ each). Note the differing vertical scales: left panel, overhead phases; right panel, Blender Render phase. Error bars show ± 1 SD.

Baseline power. The [BASELINE] phase (60 s in both arms) consumes 0.096 Wh under Design 2 against 0.076 Wh under Design 1, i.e. 26 % more idle energy at equal duration (Figure 2); at ± 1 SD the two intervals overlap. On Windows 11 client systems Hyper-V uses the *root scheduler* by default, delegating all logical-processor scheduling to the Windows NT scheduler [Mi25], so background services compete for the same physical cores in both arms. The difference therefore reflects how much background activity each measurement session carried rather than a scheduling property of the designs; see Section 5.2.

Render duration. The render takes 29 ± 9 s (10.9 %) longer under Design 2 than under Design 1 for the same binary on the same physical CPU. Since rendering is CPU-bound, any CPU time taken by concurrent background processes directly prolongs it, which is consistent with the elevated baseline observed above.

Total energy. Design 2 consumes 1.272 ± 0.028 Wh for the Blender Render phase against 1.166 ± 0.019 Wh for Design 1 – a difference of +0.106 Wh (+9.1 %). Both arms show comparable average `cpu_package` power during the render (≈ 15.5 W and ≈ 15.7 W respectively), so the extra energy is explained almost entirely by the longer render duration (29 s at ≈ 15.5 W ≈ 0.12 Wh); the marginally higher Design 1 render power narrows the gap slightly.

Net energy is reported mainly as a caution: Design 2 (0.797 ± 0.072 Wh) is marginally *lower* than Design 1 (0.825 ± 0.102 Wh), because the higher idle level contributes a larger subtracted term (≈ 0.47 vs. ≈ 0.34 Wh). Net energy thus conflates workload energy with duration and baseline effects, making total phase energy the more interpretable metric here.

5 Discussion

5.1 Evaluation of the Results

Both designs produce stable, reproducible measurements, with total-energy CVs of at most 2.7 % in every domain. Design 1 shows lower variability for `cpu_package` (1.6 % vs. 2.2 %) and more stable render durations (0.8 % vs. 3.0 %). Net-energy CVs are elevated in both, driven by run-to-run variation in idle-phase power rather than measurement noise – consistent with the session-driven baseline variance reported by Ournani et al. [OBR20]. For Design 1 this is attributable almost entirely to run 476c3d76 (host baseline ≈ 8.5 W vs. ≈ 4.1 W typical); excluding it yields a net `cpu_package` CV of 0.9 %.

As an inbox interface requiring no custom kernel driver, EMI is the more deployment-friendly option where package-level granularity suffices, while Scaphandre/MSR remains necessary for `cpu_cores` and `psys`. Architecturally Design 2 is the more direct of the two; Design 1 shows lower variability, but its runs were also the quieter evening sessions (Section 5.2), so this cannot be attributed to the hosting design.

From an EMIS perspective, two findings are relevant. First, for CPU-bound workloads on Intel single-socket hardware the architecture delivers plausible, internally consistent energy data, extending GMT's comparability framework to Windows-based corporate workstations. Second, the measurement environment is not energy-neutral: the same application on the same hardware differed by 9.1 % in measured render energy depending only on how the run was orchestrated and how much background activity the host carried. For organizations benchmarking or eco-labelling Windows software, controlling that environment is therefore as relevant as the instrument.

5.2 Limitations

The Scaphandre kernel driver is unsigned and requires Windows test-signing mode, preventing production use without an Extended Validation certificate; EMI is a viable alternative at the package level. Only single-socket systems are supported (`cpuIndex = 0`). The effective sampling interval averages ≈ 103 ms against the 99 ms target, with outliers up to 130 ms; since energy is obtained by differencing cumulative counters, this affects the resolution of the power traces but not the reported energies. `psys` covers the entire platform including non-CPU components, making `cpu_package` and `cpu_cores` more informative for workload-specific analyses.

Design 2 removes the WSL2 dependency of the GMT runner, but Docker Desktop on Windows 11 may itself use a WSL2 backend, which was not recorded, so the container runtime may retain it.

No external ground-truth validation was performed: the agreement between Scaphandre/MSR and EMI to within 0.1 % shows that two independently implemented code paths read the

same hardware counter consistently, not that the counter reflects true energy draw. RAPL is well validated against physical power meters on Linux [Kh18], but this Windows code path has not, to our knowledge, been validated against an external reference.

The comparison in Section 4.4 is subject to a time-of-day confound: the Design 2 runs were conducted during morning hours, when background services are more active, and the Design 1 runs during evening hours. Since background activity is also the proposed explanation for the observed difference, the two cannot be separated from the present data, and the 9.1 % figure should be read as an observation in the tested configuration rather than as a property of either design. Disentangling them requires repeating both sets interleaved within a single session. Reported differences are point estimates without inferential statistics. Finally, all evaluations used a single CPU model (Intel Core i7-1185G7) and a sustained CPU-bound workload, so the findings are only partially generalizable to other CPU architectures or workload classes.

6 Conclusion and Outlook

This work integrates Intel RAPL energy measurement into GMT for native Windows applications – a capability GMT previously lacked – and compares a WSL2-bridged against a native Windows hosting design, without modifying core GMT components. Ten-run studies of both designs confirm stable, internally consistent measurements, with total-energy CVs of 1.6 % and 2.2 % for `cpu_package`, and a second acquisition path agreeing to within 0.1 %. Between the two designs the identical workload differed by 9.1 % in render energy; as the two sets of runs were collected at different times of day, this is reported as observed rather than as a property of either design. Either way it is a caution for cross-environment benchmarking: the conditions of measurement can account for a share of the energy attributed to a workload.

These results extend GMT’s applicability within corporate environmental information systems to Windows-based environments, addressing UN Sustainable Development Goals 9 and 12 by making the energy footprint of Windows software measurable within a certification-anchored framework. Future work includes signed driver distribution, multi-socket support, validation on further CPU architectures such as AMD, non-CPU-bound workload classes, external validation against a physical reference meter, and repeating both studies interleaved within a single session.

Availability and Acknowledgements

The GMT metric provider and the run configuration, with all component versions pinned, are available at `<repository URL>`. This work was funded by the Deutsche Bundesstiftung Umwelt (DBU) as part of the CASO project.

References

- [Fi24] Fieni, G. et al.: PowerAPI: A Python framework for building software-defined power meters. *Journal of Open Source Software* 9 (98), p. 6670, 2024.
- [Gr24] Green Coding Solutions: Introduction, 2024, <https://docs.green-coding.io/docs/prologue/introduction/>, accessed: 01/01/2025.
- [Ho25] Hoffmann, G.-D.: Green Metrics Tool: Measuring for fun and profit, 2025, <https://arxiv.org/html/2506.23967v1>, accessed: 01/01/2025.
- [Kh18] Khan, K. N. et al.: RAPL in Action: Experiences in Using RAPL for Power Measurements. *ACM Transactions on Modeling and Performance Evaluation of Computing Systems* 3 (2), 9:1–9:26, 2018.
- [Mi25] Microsoft Corporation: Manage Hyper-V Hypervisor Scheduler Types, 2025, <https://learn.microsoft.com/en-us/windows-server/virtualization/hyper-v/manage/manage-hyper-v-scheduler-types>, accessed: 08/08/2026.
- [Na11] Naumann, S. et al.: The GREENSOFT Model: A reference model for green and sustainable software and its engineering. *Sustainable Computing: Informatics and Systems* 1 (4), pp. 294–304, 2011.
- [NGK20] Naumann, S.; Guldner, A.; Kern, E.: The Eco-label Blue Angel for Software – Development and Components. In: *Advances and New Trends in Environmental Informatics – Digital Twins for Sustainability*. Progress in IS, Springer, Cham, pp. 79–89, 2020.
- [OBR20] Ournani, Z.; Belgaid, M. C.; Rouvoy, R.: Taming Energy Consumption Variations in Systems Benchmarking. In: *Proceedings of the 11th ACM/SPEC International Conference on Performance Engineering (ICPE)*. ACM, pp. 58–69, 2020.
- [RT24] Raffin, G.; Trystram, D.: Dissecting the software-based measurement of CPU energy consumption: a comparative analysis, 2024, <https://arxiv.org/abs/2401.15985>, accessed: 01/01/2025.
- [Sc19] Schöne, R. et al.: Energy Efficiency Features of the Intel Skylake-SP Processor and Their Impact on Performance. In: *2019 International Conference on High Performance Computing & Simulation (HPCS)*. IEEE, Dublin, pp. 399–406, 2019.
- [Sc24] Scaphandre Project: Scaphandre: A Metrology Agent for Energy Consumption, 2024, <https://github.com/hubblo-org/scaphandre>, accessed: 01/01/2025.
- [SCD24] Sallou, J.; Cruz, L.; Durieux, T.: EnergiBridge: Empowering Software Sustainability through Cross-Platform Energy Measurement. In: *Proceedings of the 46th International Conference on Software Engineering: Companion Proceedings (ICSE Companion)*. ACM, 2024.
- [Wo05] Wohlgemuth, V.: *Betriebliche Umweltinformationssysteme*. Springer, Berlin, Heidelberg, 2005.